

Exercise 1: Mocking and Stubbing

Scenario: You need to test a service that depends on an external API. Use Mockito to mock the external API and stub its methods.

Code:

ExternalApi.java

```
package com.Mockito;

public interface ExternalApi {

    String getData();

}
```

MyService.java

```
package com.Mockito;

public class MyService {

    private ExternalApi api;

    public MyService(ExternalApi api) {
        this.api = api;
    }

    public String fetchData() {
        return api.getData();
    }

}
```

MyServiceTest.java

```
package com.Mockito;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.when;

import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MyServiceTest {

    @Test
    public void testExternalApi() {

        // Create a mock object
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);
```

```

    // Stub the method
    when(mockApi.getData()).thenReturn("Mock Data");

    // Create service object
    MyService service = new MyService(mockApi);

    // Fetch data
    String result = service.fetchData();

    // Verify the result
    assertEquals("Mock Data", result);
}
}

```

Output:

The screenshot shows an IDE with the following components:

- Project Explorer:** Shows the project structure with a package `com.Mockito` containing `ExternalApi` and `MyService`. A test class `MyServiceTest` is located in the `test` directory.
- Editor:** Displays the `MyServiceTest.java` file, which contains the test code for `testExternalApi()`. The code includes imports for Mockito and JUnit, and uses `Mockito.mock()` and `Mockito.when()` to set up the test.
- Run Console:** Shows the output of the test run. It indicates that the test `testExternalApi()` passed successfully, with a duration of 949 ms. The console also shows the JVM command used to run the test and a warning about the bootstrap classpath.

Exercise 2: Verifying Interactions

Scenario: You need to ensure that a method is called with specific arguments.

Code:

ExternalAPI.java

```

package com.Mockito;

public interface ExternalApi {

    String getData();

}

```

MyService.java

```
package com.Mockito;

public class MyService {

    private ExternalApi api;

    public MyService(ExternalApi api) {
        this.api = api;
    }

    public String fetchData() {
        return api.getData();
    }

}
```

MyServiceTest.java

```
package com.Mockito;

import static org.mockito.Mockito.verify;

import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MyServiceTest {

    @Test
    public void testVerifyInteraction() {

        // Create a mock object
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);

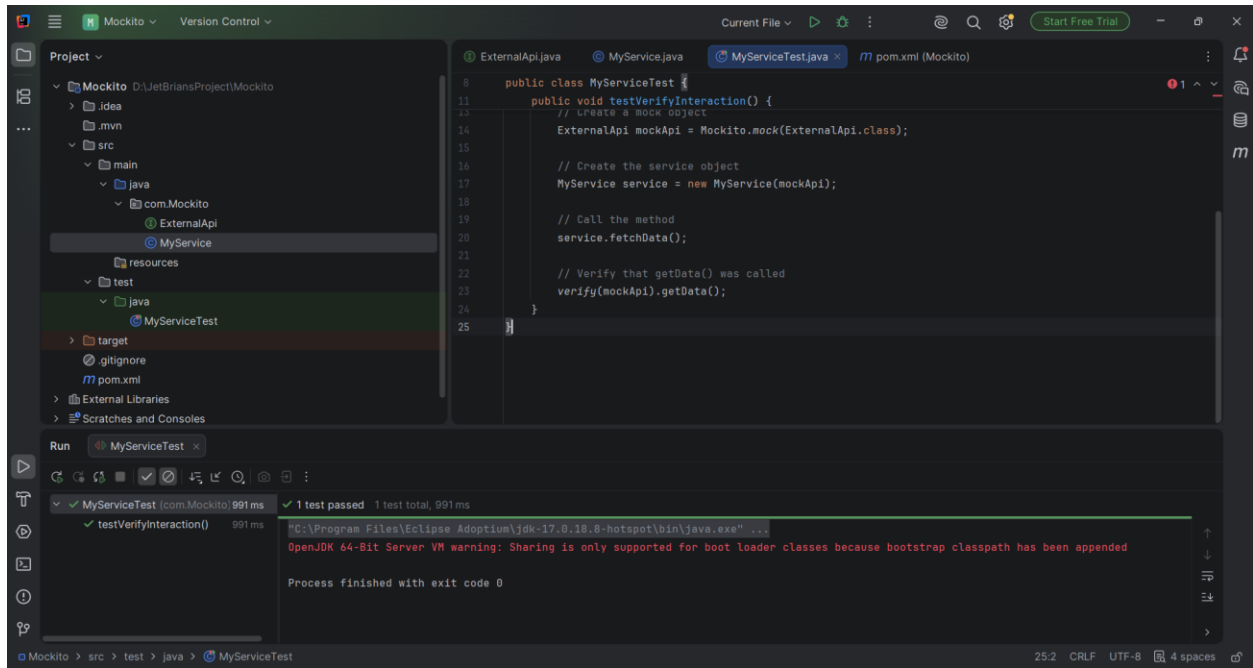
        // Create the service object
        MyService service = new MyService(mockApi);

        // Call the method
        service.fetchData();

        // Verify that getData() was called
        verify(mockApi).getData();
    }

}
```

Output:



Exercise 3: Argument Matching

Scenario: You need to verify that a method is called with specific arguments.

ExternalApi.java

```
package com.Mockito;

public interface ExternalApi {

    String getData(String userId);

}
```

MyService.java

```
package com.Mockito;

public class MyService {

    private ExternalApi api;

    public MyService(ExternalApi api) {
        this.api = api;
    }

    public String fetchData(String userId) {
        return api.getData(userId);
    }

}
```

MyServiceTest.java

```
package com.Mockito;

import static org.mockito.ArgumentMatchers.anyString;
import static org.mockito.Mockito.verify;

import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MyServiceTest {

    @Test
    public void testArgumentMatching() {

        // Create a mock object
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);

        // Create the service object
        MyService service = new MyService(mockApi);

    }
```

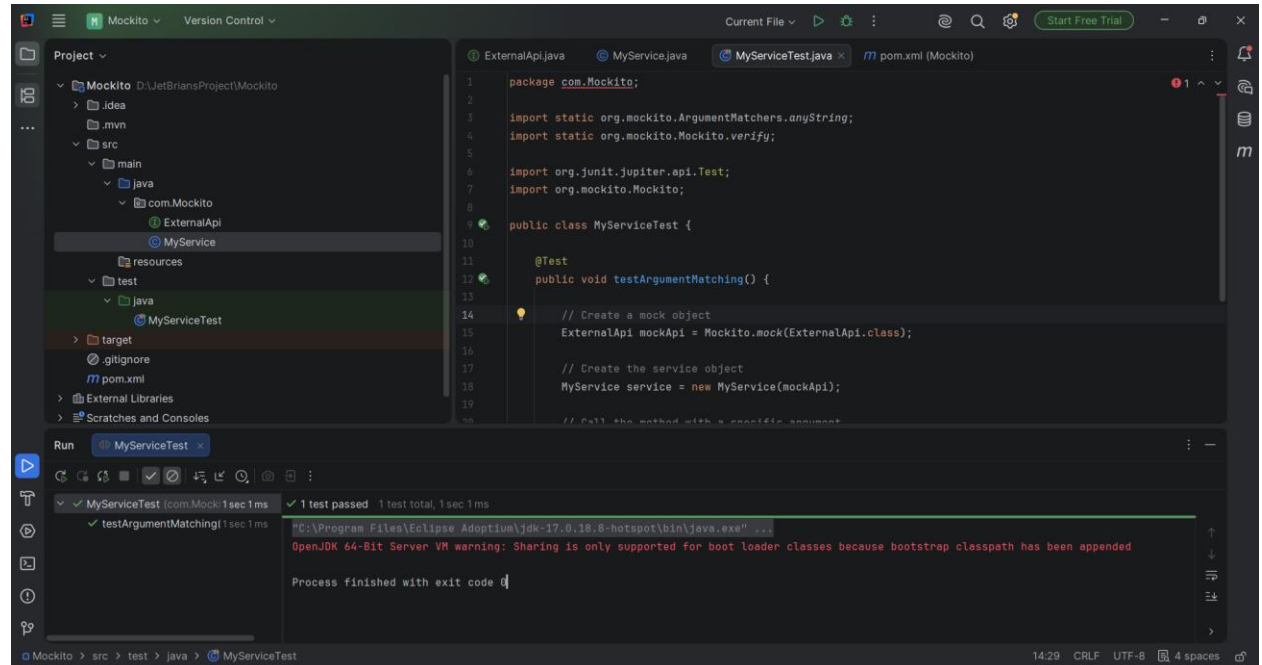
```

    // Call the method with a specific argument
    service.fetchData("User123");

    // Verify that getData() was called with any String argument
    verify(mockApi).getData(anyString());
}

```

Output:



Exercise 4: Handling Void Methods

Scenario: You need to test a void method that performs some action.

Code:

ExternalApi.java

```

package com.Mockito;

public interface ExternalApi {

    void sendData();

}

```

MyService.java

```

package com.Mockito;

```

```
public class MyService {

    private ExternalApi api;

    public MyService(ExternalApi api) {
        this.api = api;
    }

    public void processData() {
        api.sendData();
    }

}
```

MyServiceTest.java

```
package com.Mockito;

import static org.mockito.Mockito.doNothing;
import static org.mockito.Mockito.verify;

import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MyServiceTest {

    @Test
    public void testVoidMethod() {

        // Create a mock object
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);

        // Stub the void method
        doNothing().when(mockApi).sendData();

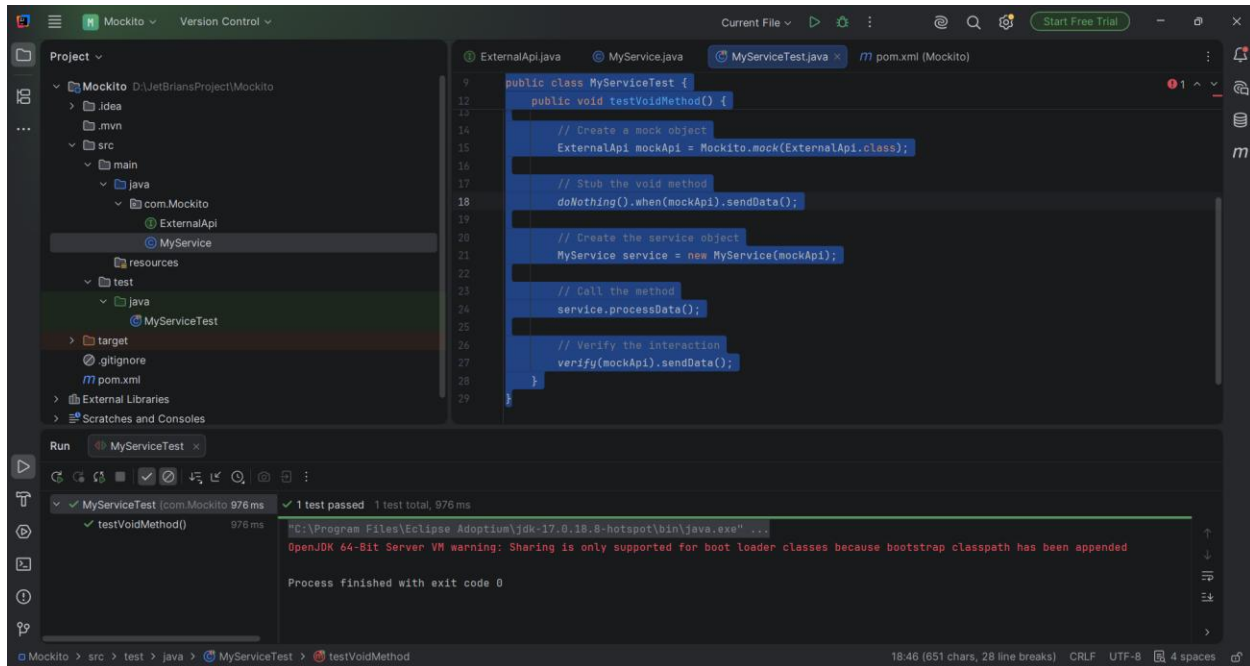
        // Create the service object
        MyService service = new MyService(mockApi);

        // Call the method
        service.processData();

        // Verify the interaction
        verify(mockApi).sendData();
    }

}
```

Output:



Exercise 5: Mocking and Stubbing with Multiple Returns

Scenario: You need to test a service that depends on an external API with multiple return values.

Code:

ExternalApi.java

```
package com.Mockito;

public interface ExternalApi {

    String getData();

}
```

MyService.java

```
package com.Mockito;

public class MyService {

    private ExternalApi api;

    public MyService(ExternalApi api) {
        this.api = api;
    }

}
```

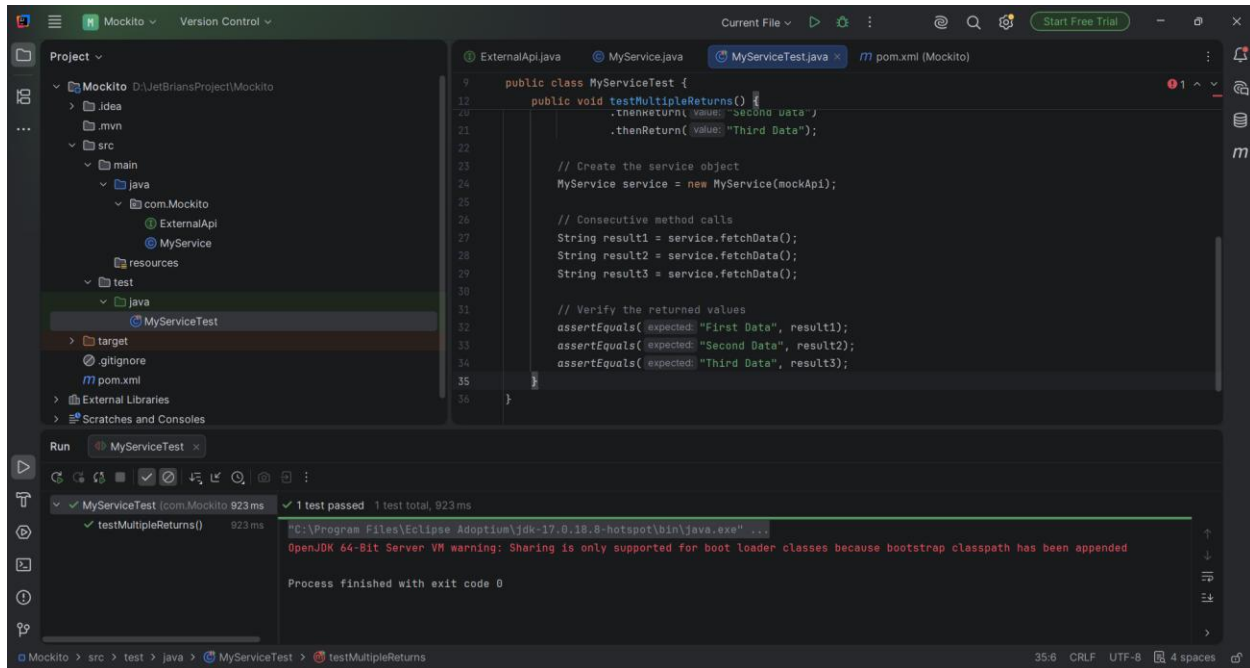


```
    public String fetchData() {  
        return api.getData();  
    }  
}
```

MyServiceTest.java

```
package com.Mockito;  
  
import static org.junit.jupiter.api.Assertions.assertEquals;  
import static org.mockito.Mockito.when;  
  
import org.junit.jupiter.api.Test;  
import org.mockito.Mockito;  
  
public class MyServiceTest {  
  
    @Test  
    public void testMultipleReturns() {  
  
        // Create a mock object  
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);  
  
        // Stub the method with multiple return values  
        when(mockApi.getData())  
            .thenReturn("First Data")  
            .thenReturn("Second Data")  
            .thenReturn("Third Data");  
  
        // Create the service object  
        MyService service = new MyService(mockApi);  
  
        // Consecutive method calls  
        String result1 = service.fetchData();  
        String result2 = service.fetchData();  
        String result3 = service.fetchData();  
  
        // Verify the returned values  
        assertEquals("First Data", result1);  
        assertEquals("Second Data", result2);  
        assertEquals("Third Data", result3);  
    }  
}
```

Output:



The screenshot displays an IDE interface with the following components:

- Project Explorer:** Shows a project named 'Mockito' with a directory structure including 'src/main/java/com.Mockito', 'resources', and 'test/java'. The 'MyServiceTest' class is highlighted in the 'test/java' directory.
- Code Editor:** Displays the source code of 'MyServiceTest.java'. The code includes a test method 'testMultipleReturns()' that uses Mockito's 'thenReturn()' to simulate a service returning three different data strings ('First Data', 'Second Data', 'Third Data'). It also includes assertions to verify the returned values.
- Run Console:** Shows the execution results of the test. It indicates that '1 test passed' and '1 test total, 923 ms'. The console output includes the Java command used to run the test and a warning from the OpenJDK 64-Bit Server VM regarding classpath sharing.

```
public class MyServiceTest {  
    public void testMultipleReturns() {  
        .thenReturn("Second Data");  
        .thenReturn("Third Data");  
  
        // Create the service object  
        MyService service = new MyService(mockApi);  
  
        // Consecutive method calls  
        String result1 = service.fetchData();  
        String result2 = service.fetchData();  
        String result3 = service.fetchData();  
  
        // Verify the returned values  
        assertEquals("First Data", result1);  
        assertEquals("Second Data", result2);  
        assertEquals("Third Data", result3);  
    }  
}
```

Run: MyServiceTest (com.Mockito 923 ms) ✓ 1 test passed 1 test total, 923 ms

testMultipleReturns() 923 ms

Process finished with exit code 0

Exercise 6: Verifying Interaction Order

Scenario: You need to ensure that methods are called in a specific order.

Code:

ExternalApi.java

```
package com.Mockito;

public interface ExternalApi {

    void connect();

    void disconnect();

}
```

MyService.java

```
package com.Mockito;

public class MyService {

    private ExternalApi api;

    public MyService(ExternalApi api) {
        this.api = api;
    }

    public void performOperation() {
        api.connect();
        api.disconnect();
    }

}
```

MyServiceTest.java

```
package com.Mockito;

import org.junit.jupiter.api.Test;
import org.mockito.InOrder;
import org.mockito.Mockito;

public class MyServiceTest {

    @Test
    public void testInteractionOrder() {

        // Create a mock object
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);

    }
```

```

// Create the service object
MyService service = new MyService(mockApi);

// Call the methods in a specific order
service.performOperation();

// Verify the interaction order
InOrder inOrder = Mockito.inOrder(mockApi);

inOrder.verify(mockApi).connect();
inOrder.verify(mockApi).disconnect();
}
}

```

Output:

The screenshot shows an IDE with a project named 'Mockito'. The project structure includes a 'test' directory with a 'MyServiceTest' class. The code editor displays the following Java code for 'MyServiceTest.java':

```

7 public class MyServiceTest {
8     public void testInteractionOrder() {
9         // Create the service object
10        MyService service = new MyService(mockApi);
11
12        // Call the methods in a specific order
13        service.performOperation();
14
15        // Verify the interaction order
16        InOrder inOrder = Mockito.inOrder(mockApi);
17
18        inOrder.verify(mockApi).connect();
19        inOrder.verify(mockApi).disconnect();
20    }
21 }

```

The Run console shows the test execution results:

```

Run MyServiceTest x
✓ MyServiceTest (com.Mockito 889 ms) ✓ 1 test passed 1 test total, 889 ms
✓ testInteractionOrder() 889 ms
"C:\Program Files\Eclipse Adoptium\jdk-17.0.18-hotspot\bin\java.exe" ...
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
Process finished with exit code 0

```

Exercise 7: Handling Void Methods with Exceptions

Scenario: You need to test a void method that throws an exception.

Code:

ExternalApi.java

```

package com.Mockito;

public interface ExternalApi {

    void sendData();

}

```

MyService.java

```
package com.Mockito;

public class MyService {

    private ExternalApi api;

    public MyService(ExternalApi api) {
        this.api = api;
    }

    public void processData() {
        api.sendData();
    }

}
```

MyServiceTest.java

```
package com.Mockito;

import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.mockito.Mockito.doThrow;
import static org.mockito.Mockito.verify;

import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MyServiceTest {

    @Test
    public void testVoidMethodThrowsException() {

        // Create a mock object
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);

        // Stub the void method to throw an exception
        doThrow(new RuntimeException("API Error"))
            .when(mockApi)
            .sendData();

        // Create the service object
        MyService service = new MyService(mockApi);

        // Verify that the exception is thrown
        assertThrows(RuntimeException.class, () -> {
            service.processData();
        });

        // Verify the interaction
        verify(mockApi).sendData();
    }

}
```

Output:

The screenshot displays an IDE interface with a project named 'Mockito'. The project structure on the left includes 'src', 'main', 'test', and 'target' directories. The 'test' directory contains a 'java' subdirectory with the file 'MyServiceTest'. The main editor shows the code for 'MyServiceTest.java', which includes a test method 'testVoidMethodThrowsException()' that creates a 'MyService' object, calls 'processData()', and verifies that a 'RuntimeException' is thrown. The 'Run' tab at the bottom shows the test execution results: 'MyServiceTest (com.Mockito) 899 ms' and 'testVoidMethodThrowsEx 899 ms', both marked as passed. The output console shows the Java command used to run the test and a warning from OpenJDK 64-Bit Server VM regarding classpath sharing, followed by the message 'Process finished with exit code 0'.

```
public class MyServiceTest {  
    public void testVoidMethodThrowsException() {  
        // Create the service object  
        MyService service = new MyService(mockApi);  
        // Verify that the exception is thrown  
        assertThrows(RuntimeException.class, () -> {  
            service.processData();  
        });  
        // Verify the interaction  
        verify(mockApi).sendData();  
    }  
}
```

Run: MyServiceTest x

✓ MyServiceTest (com.Mockito) 899 ms
✓ testVoidMethodThrowsEx 899 ms

1 test passed 1 test total, 899 ms

"C:\Program Files\Eclipse Adoptium\jdk-17.0.18-hotspot\bin\java.exe" ...
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

Process finished with exit code 0

Mockito > src > test > java > MyServiceTest > testVoidMethodThrowsException 29:12 (901 chars, 33 line breaks) CRLF UTF-8 4 spaces